

Universidad Diego Portales

Facultad de Ingeniería y Ciencias

Curso Arquitectura de Software

Sistema de Gestión de Inventario Omnicanal (Sync-Stock)

Informe Final - 3ra Entrega

Organización: Tienda de Pesca "PescaTotal"

Área: Departamento de Ventas e Inventario

Responsables: Fernando Llancao, Carlos Jara, Bastian Peña, Nicolas Canelo

Fecha: 7 de Junio de 2026

Ubicación: Región Metropolitana, Chile

Semestre 1/2026

Índice

1. Equipo de Trabajo	3
2. Descripción del Sistema y Contexto Organizacional	3
2.1. La Organización: PescaTotal	3
2.2. Área de Trabajo: Departamento de Ventas e Inventario	3
2.3. Problemática Actual	3
2.4. Sistema Propuesto: Sync-Stock	4
3. Objetivos y Perfiles de Usuario	4
3.1. Objetivos del Sistema	4
3.1.1. Objetivo General	4
3.1.2. Objetivos Específicos	4
3.2. Perfiles de Usuario	4
4. Requerimientos Funcionales	5
5. Modelo de Datos y Mecanismo de Persistencia	7
5.1. Mecanismo de Persistencia y Decisiones de Diseño	7
5.2. Diccionario de Datos (Entidades del Sistema Omnicanal)	7
6. Arquitectura SOA y Mediación a través de un Bus de Servicios (ESB)	8
6.1. El Rol del Enterprise Service Bus (ESB) y Límite Arquitectónico	8
6.2. Mecanismos Clásicos de Mediación SOA Implementados	10
6.3. Componentes de la Arquitectura	11
6.3.1. Componentes Cliente (Consumidores de Servicios)	11
6.3.2. Componentes de Servicio (Proveedores del Bus)	11
7. Interfaces de los Componentes y Contratos de Mensajería TCP	11
7.1. Detalle de Contratos de Mensajería TCP	12
7.1.1. 1. Servicio de Autenticación (Servicio <code>autho</code>)	12
7.1.2. 2. Reserva Temporal de Stock (Servicio <code>inven</code> , Acción <code>reserve</code>) . .	13
7.1.3. 3. Confirmación de Reserva (Servicio <code>inven</code> , Acción <code>commit</code>)	13
7.1.4. 4. Consulta de Stock en Tiempo Real (Servicio <code>inven</code> , Acción <code>stock</code>)	14
8. Documentación de Servicios y Procesos de Mensajería TCP	14
8.1. Registro de Nuevos Productos	15
8.2. Modificación de Producto	15
8.3. Despacho de Órdenes Consolidadas	16
8.4. Redirección General a través del Bus de Servicios (Acciones de Lectura) . .	17
9. Documentación del Sistema y Evidencias de Funcionamiento	18
9.1. Explicación Técnica del Código e Implementación	18
9.2. Evidencias de Funcionamiento y Logs	18

10. Análisis Crítico	20
10.1. Latencia y Concurrencia de los Bloqueos Pesimistas	20
10.2. Desafíos del Desarrollo de un ESB Personalizado TCP	20
10.3. Seguridad y RBAC en Entornos Distribuidos	20
10.4. Control de Inactividad y Seguridad Física en Tienda	20
11. Conclusiones	21

1. Equipo de Trabajo

Para el desarrollo del sistema Sync-Stock, se ha conformado un equipo de trabajo liderado por el responsable del área de desarrollo:

Integrante	Cargo	Responsabilidades
Fernando Llancao	CEO / Líder de Proyecto	■ Liderazgo estratégico y toma de decisiones ejecutivas. ■ Coordinación directa con los dueños de PescaTotal. ■ Supervisión general del análisis y diseño del sistema.
Nicolas Canelo	Arquitecto de Software	■ Diseño de la estructura técnica y componentes. ■ Selección del stack tecnológico adecuado para tiempo real.
Carlos Jara	Analista de Sistemas	■ Levantamiento y especificación de requerimientos. ■ Definición del modelo de datos y persistencia.
Bastian Peña	Desarrollador Full-stack	■ Implementación de la lógica de negocio y API. ■ Desarrollo de la interfaz de usuario amigable.

2. Descripción del Sistema y Contexto Organizacional

2.1. La Organización: PescaTotal

PescaTotal es una tienda retail especializada en artículos de pesca recreativa y equipamiento outdoor. A pesar de ser una organización de escala pequeña, posee un flujo de ventas constante y una base de clientes fidelizada en la Región Metropolitana.

2.2. Área de Trabajo: Departamento de Ventas e Inventario

Esta unidad operativa gestiona el stock físico de la tienda y, recientemente, la administración del nuevo canal e-commerce. Sus funciones incluyen la recepción de suministros, atención al público y preparación de pedidos online.

2.3. Problemática Actual

La tienda enfrenta dificultades operacionales críticas derivadas de la falta de integración digital:

- **Incumplimiento de pedidos y fricción con clientes por falta de gestión de reservas:** Al carecer de un mecanismo que bloquee o separe físicamente los productos apenas se venden por el canal e-commerce, los artículos quedan expuestos en las vitrinas y son comprados por clientes presenciales. Esto genera un problema grave para el negocio: la necesidad de cancelar compras online ya pagadas, procesar devoluciones de dinero, sobrecargar al personal de atención al cliente y generar un impacto negativo en la reputación y fidelidad de los compradores de PescaTotal.

2.4. Sistema Propuesto: Sync-Stock

Sync-Stock es una plataforma segura de gestión de inventario omnicanal. Su objetivo principal es actuar como un núcleo de datos centralizado que sincronice el stock en tiempo real entre la tienda física y el e-commerce.

Para garantizar la integridad de las operaciones, el sistema contará con acceso restringido mediante roles, entregando a cada empleado (Vendedor, Encargado E-commerce o Administrador) solo las herramientas que necesita. Al registrar una venta en el mesón o recibir un pedido web, el sistema no solo actualiza la disponibilidad global inmediatamente, sino que genera un estado de Reservainmutable para las compras web, bloqueando la posibilidad de sobreventas en la tienda física. Esto asegurará la coherencia total de los datos y protegerá la experiencia de compra de los clientes.

3. Objetivos y Perfiles de Usuario

3.1. Objetivos del Sistema

3.1.1. Objetivo General

Implementar un sistema integral que garantice la coherencia total del inventario entre la tienda física y el e-commerce de PescaTotal, optimizando la eficiencia operativa y eliminando errores de sobreventa.

3.1.2. Objetivos Específicos

1. Sincronizar de forma automática y bidireccional el stock tras cada transacción.
2. Implementar un módulo de reserva inmediata para asegurar productos vendidos vía web.
3. Proporcionar visibilidad de stock real al personal de ventas mediante una interfaz simple.
4. Reducir en un 85 % las horas-hombre semanales dedicadas a la conciliación y revisión manual de inventarios, alcanzando esta métrica al finalizar el segundo mes de operación en producción del sistema Sync-Stock.”

3.2. Perfiles de Usuario

- **Usuario Primario (Vendedor):** Utiliza el sistema en el mesón de la tienda para consultar disponibilidad y cerrar ventas presenciales.

- Frecuencia de uso: Permanente durante la jornada.
- Nivel tecnico: Intermedio.
- **Usuario Secundario (Encargado E-commerce):** Gestiona los pedidos web, visualiza las reservas y confirma la salida física del producto.
 - Frecuencia de uso: Permanente durante la jornada.
 - Nivel tecnico: Intermedio.
- **Usuario de Apoyo (Administrador):** Gestiona el catálogo, permisos de usuarios y configuraciones de alertas de stock.
 - Frecuencia de uso: Variable, segun necesidades de la tienda.
 - Nivel tecnico: Avanzado.

4. Requerimientos Funcionales

Código	Nombre	Descripción
RF-001	Sincronización Real-Time	El sistema debe descontar el stock del catálogo online inmediatamente tras una venta física confirmada.
RF-002	Reserva de Stock Online	Al detectarse una orden de compra web, el sistema debe mover el producto al estado Reservado”, restándolo del stock físico vendible.
RF-003	Gestión de Productos	Permitir el registro, edición y categorización de artículos con códigos de barra y ubicación física.
RF-004	Control de Despachos	Funcionalidad para confirmar la salida de productos reservados, actualizando el estado a ”Vendido” definitivamente.
RF-005	Alertas de Quiebre	Notificaciones automáticas cuando el stock disponible llegue a un umbral mínimo configurado.
RF-006	Log de Auditoría	Registro detallado de quién, cuándo y por qué canal se realizó cada movimiento de inventario.
RF-007	Autenticación de Usuarios (Login)	El sistema debe requerir que todo usuario inicie sesión mediante credenciales (correo/usuario y contraseña) antes de acceder a cualquier funcionalidad.
RF-008	Control de Acceso Basado en Roles (RBAC)	El sistema debe restringir las funcionalidades e interfaces de acuerdo con el rol del usuario (Administrador, Vendedor, Encargado E-commerce). Por ejemplo, solo el Administrador podrá gestionar permisos o el catálogo.
RF-009	Cierre de Sesión y Tiempos de Inactividad	El sistema debe permitir el cierre de sesión manual y forzar un cierre automático tras 30 minutos de inactividad por motivos de seguridad en los equipos compartidos de la tienda.

5. Modelo de Datos y Mecanismo de Persistencia

Para dar soporte a las funcionalidades del sistema Sync-Stock en un entorno retail omnicanal real, se ha diseñado una estrategia de persistencia transaccional de alta consistencia y un modelo de datos relacional detallado.

5.1. Mecanismo de Persistencia y Decisiones de Diseño

Se ha optado por implementar una **Base de Datos Relacional (RDBMS)** utilizando **PostgreSQL**. Esta decisión responde a la necesidad de resolver problemas complejos de concurrencia y consistencia que un sistema omnicanal enfrenta en tiempo real:

1. **Consistencia Transaccional Estricta (ACID):** Para evitar la sobreventa, los cambios en el inventario deben procesarse con aislamiento transaccional estricto. PostgreSQL permite aislar operaciones mediante el nivel de aislamiento de transacciones *Read Committed* combinado con bloqueos pesimistas explícitos.
2. **Estrategia de Concurrencia y Bloqueos Pesimistas (Pessimistic Locking):** Al procesar una compra en el Punto de Venta (POS) o iniciar un proceso de reserva web, el sistema ejecuta una transacción que bloquea las filas correspondientes de la tabla de inventario mediante la sentencia `SELECT ... FOR UPDATE`.
 - Esto garantiza que ningún otro proceso pueda modificar la cantidad del producto hasta que la transacción actual finalice (haga *COMMIT* o *ROLLBACK*).
 - **Prevención de Deadlocks:** Para evitar bloqueos mutuos (*deadlocks*) bajo cargas de alta concurrencia, el sistema ordena siempre los ítems de la consulta de forma determinista (ordenados por el identificador del producto) antes de solicitar el bloqueo.
3. **Política de Expiración de Reservas y Liberación de Stock (Time-To-Live):** El canal e-commerce genera reservas temporales para proteger el stock de un cliente mientras completa el pago. Sin embargo, para evitar que el stock quede bloqueado indefinidamente si el cliente abandona el carrito, se implementa una regla de expiración de 15 minutos.
 - Cada reserva se registra en la tabla **Reserva** con un campo **FechaExpiracion**.
 - Un proceso daemon (o *cron job*) en el bus de servicios ejecuta periódicamente (cada 1 minuto) una transacción que selecciona las reservas expiradas en estado **Pendiente**, revierte el stock reservado al stock disponible ($\text{StockDisponible} = \text{StockDisponible} + \text{Cantidad}$) y marca la reserva como **Expirada**.
4. **Pooling de Conexiones y Alta Disponibilidad:** Para asegurar tiempos de respuesta menores a 200 ms y soportar picos de carga concurrentes entre la tienda y el sitio web, se utiliza un pooler de conexiones (**PgBouncer**) configurado en modo transacción. Esto permite optimizar el uso de recursos y mantener la latencia en rangos óptimos.

5.2. Diccionario de Datos (Entidades del Sistema Omnicanal)

A continuación se detalla el esquema de base de datos extendido que soporta la lógica del inventario multicanal, reservas y auditoría de transacciones:

Entidad	Atributo	Tipo de Dato	Descripción
Producto	ID_Producto (PK)	UUID	Identificador universal único del artículo.
	SKU	VARCHAR(50)	Código SKU único (Stock Keeping Unit).
	Nombre	VARCHAR(150)	Nombre o descripción del artículo.
	Precio	DECIMAL(12,2)	Precio de venta unitario.
Ubicacion	ID_Ubicacion (PK)	UUID	Identificador único de la ubicación de stock.
	Nombre	VARCHAR(100)	Nombre (ej. 'Bodega Central', 'POS Tienda').
	Tipo	ENUM	Tipo de instalación: 'Tienda' o 'Bodega'.
Inventario	ID_Inventario (PK)	UUID	Identificador del registro de inventario.
	Producto_ID (FK)	UUID	Referencia a la entidad Producto .
	Ubicacion_ID (FK)	UUID	Referencia a la entidad Ubicacion .
	StockDisponible	INT	Unidades físicas disponibles para venta inmediata.
	StockReservado	INT	Unidades retenidas temporalmente por e-commerce.
	StockTransito	INT	Unidades en proceso de traslado entre bodegas.

Cuadro 1: Diccionario de datos: Catálogo, Ubicaciones e Inventario.

Entidad	Atributo	Tipo de Dato	Descripción
Reserva	ID_Reserva (PK)	UUID	Identificador único de la reserva temporal.
	Producto_ID (FK)	UUID	Referencia a la entidad Producto .
	Ubicacion_ID (FK)	UUID	Referencia a la entidad Ubicacion .
	Cantidad	INT	Cantidad de unidades bloqueadas.
	Estado	ENUM	Estado de la reserva: 'Pendiente', 'Confirmada', 'Expirada'.
	FechaCreacion	TIMESTAMP	Marca de tiempo de la creación de la reserva.
Orden	FechaExpiracion	TIMESTAMP	Límite temporal para consolidar la transacción.
	ID_Orden (PK)	UUID	Identificador único de la orden de venta.
	Canal	ENUM	Origen del pedido: 'POS_Fisico' o 'E_Commerce'.
	Estado	ENUM	Estado del pedido: 'Creada', 'Pagada', 'Despachada', 'Cancelada'.
	FechaCreacion	TIMESTAMP	Marca de tiempo de registro de la orden.
LineaOrden	Total	DECIMAL(12,2)	Monto total de la transacción comercial.
	ID_Linea (PK)	UUID	Identificador de la línea de detalle de la orden.
	Orden_ID (FK)	UUID	Referencia a la entidad Orden .
	Producto_ID (FK)	UUID	Referencia a la entidad Producto .
	Cantidad	INT	Cantidad de unidades compradas.
	PrecioUnitario	DECIMAL(12,2)	Precio facturado para este producto.

Cuadro 2: Diccionario de datos: Reservas temporales y Órdenes de venta.

Entidad	Atributo	Tipo de Dato	Descripción
HistorialInv	ID_Historial (PK)	UUID	Identificador único para auditoría de movimientos.
	Producto_ID (FK)	UUID	Referencia al producto afectado.
	Ubicacion_ID (FK)	UUID	Ubicación física del movimiento.
	Cantidad	INT	Unidades modificadas (valor positivo o negativo).
	TipoMovimiento	ENUM	Tipo de evento: 'Venta', 'Reserva_Creada', 'Reserva_Expirada', 'Recepcion', 'Ajuste'.
	FechaHora	TIMESTAMP	Fecha y hora exacta de la transacción.
	Usuario_ID (FK)	UUID	Identificador del operario responsable.

Cuadro 3: Diccionario de datos: Historial de inventario para log de auditoría.

6. Arquitectura SOA y Mediación a través de un Bus de Servicios (ESB)

Para resolver la integración y sincronización en tiempo real de múltiples canales, el sistema se ha estructurado bajo el patrón **SOA (Service-Oriented Architecture)** incorporando un **Enterprise Service Bus (ESB)** central como columna vertebral de comunicación. El ESB centraliza las interacciones y proporciona mecanismos de mediación clásicos que desacoplan físicamente y lógicamente a los clientes y servicios, operando de manera exclusiva en la capa de transporte mediante conexiones TCP persistentes.

6.1. El Rol del Enterprise Service Bus (ESB) y Límite Arquitectónico

A fin de asegurar el cumplimiento de los estándares de una arquitectura SOA tradicional y evitar el uso de protocolos acoplados a la web dentro del bus de servicios, se ha establecido un límite arquitectónico estricto. ****No existe presencia de HTTP ni de APIs REST dentro del núcleo de la arquitectura SOA.**** La comunicación entre el ESB y los microservicios proveedores, así como con los consumidores nativos, se realiza de forma exclusiva mediante sockets TCP utilizando un protocolo de mensajería con entramado de longitud fija (*Length-Prefixed Framing*).

El ESB proporciona las siguientes capacidades de infraestructura:

- **Garantía de Entrega y Asincronía:** Permite encolar las peticiones en canales de mensajería TCP dedicados. Si un servicio (como el de inventario o despacho) experimenta una indisponibilidad temporal, los mensajes TCP quedan retenidos en el bus en un buffer de conexión hasta que el servicio correspondiente restablezca su socket y procese el mensaje, garantizando que no se pierdan transacciones de negocio.
- **Desacoplamiento Tecnológico y de Protocolos:** Al independizar la capa de servicios del protocolo HTTP, la arquitectura SOA puede integrar de forma nativa microservicios construidos en diversos lenguajes y plataformas (como scripts Python de automatización o servicios de backend en Node.js) que se comunican directamente a través de flujos TCP estructurados (JSON/TCP), eliminando la sobrecarga y la dependencia de servidores web tradicionales.
- **Gateway/Adaptador de Clientes Web Externos (Perímetro):** Debido a las restricciones de seguridad de los navegadores web modernos, las aplicaciones frontend en ejecución en el cliente (como React) no pueden abrir sockets TCP crudos directamente. Para solventar esto, se posiciona un ****API Gateway / Adaptador de Clientes (Express.js)**** en la frontera exterior del sistema. Este adaptador actúa como un cliente externo a la red SOA: recibe las llamadas HTTP REST del navegador web, las valida y autoriza (RBAC mediante tokens JWT) y las traduce a mensajes en formato JSON/TCP que inyecta en el ESB. De esta forma, el protocolo HTTP queda estrictamente relegado al perímetro de red externa, fuera del núcleo de servicios SOA.

6.2. Mecanismos Clásicos de Mediación SOA Implementados

El ESB actúa como una capa de mediación activa sobre los flujos TCP de datos, aplicando patrones de integración empresarial (EIP) clásicos para procesar los mensajes en tránsito:

1. **Enrutamiento Basado en Contenido (Content-Based Router):** Cuando el ESB recibe una trama TCP de orden o consulta, inspecciona el campo `action` o `canal` del payload.
 - Si el mensaje proviene de un canal digital como `E.Commerce`, el ESB enruta dinámicamente el mensaje hacia el servicio de inventario para la validación de reservas y flujo de pago.
 - Si el mensaje proviene de un canal físico como el punto de venta `POS.Fisico`, es enrutado directamente al servicio de inventario para una rebaja directa sin estado intermedio de reserva.
2. **Transformación y Traducción de Datos (Data Transformer):** Para homogeneizar los formatos de las peticiones que ingresan al bus (como payloads provenientes de terminales POS antiguas o plataformas e-commerce con campos abreviados), el mediador aplica un transformador que normaliza los campos de entrada al esquema estandarizado del bus (*Modelo Canónico*) antes de que el mensaje llegue al servicio de inventario.
3. **Enriquecimiento de Mensajes (Message Enricher):** Cuando un cliente envía una trama TCP simplificada (con el SKU y cantidad únicamente), el bus intercepta el mensaje y consulta internamente al catálogo para enriquecer el payload agregando el nombre del producto, descripción y precio unitario vigente. Esto asegura que los servicios core reciban mensajes completos y contextualizados para su procesamiento.
4. **Puente de Protocolos (Protocol Bridge) de Frontera:** A nivel perimetral, la pasarela de servicios traduce las peticiones HTTP REST síncronas del frontend web en tramas estructuradas TCP enviadas al bus. Este componente de frontera permite la interoperabilidad de la interfaz de usuario con la arquitectura SOA interna, traduciendo formatos, cabeceras y parámetros HTTP a sockets de red TCP y viceversa.

6.3. Componentes de la Arquitectura

Bajo este esquema de mediación, los componentes del ecosistema SOA se dividen en:

6.3.1. Componentes Cliente (Consumidores de Servicios)

- **Cientes Directos TCP (POS de Escritorio / Scripts de Automatización):** Clientes de escritorio y procesos automatizados que se conectan directamente al ESB en el puerto 5000 utilizando sockets TCP crudos, consumiendo servicios de manera nativa sin intermediarios HTTP.
- **API Gateway de Frontera (Adaptador REST-TCP):** Servidor web Express que actúa como puente de frontera. Traduce las llamadas HTTP REST del navegador web (Frontend POS, Dashboard de Administración) a mensajes de red TCP compatibles con el bus SOA.

6.3.2. Componentes de Servicio (Proveedores del Bus)

- **Auth Service (Servicio 'autho'):** Servicio de autenticación registrado en el bus TCP. Procesa credenciales de usuario y genera firmas criptográficas JWT.
- **Inventory Service (Servicio 'inven'):** Servicio core de inventario. Resuelve consultas y realiza reservas o confirmaciones de stock interactuando de forma atómica con la base de datos PostgreSQL.
- **Sales & Order Service (Servicio 'sales'):** Gestiona el flujo de órdenes, coordinando con el servicio de inventario a través de mensajes TCP y auditando las transacciones de negocio.

7. Interfaces de los Componentes y Contratos de Mensajería TCP

Para garantizar la interoperabilidad del sistema Sync-Stock en un entorno omnicanal crítico en tiempo real, se definen contratos de mensajería TCP rigurosos a nivel del bus de servicios (ESB). Cada contrato especifica el identificador del servicio destino, el tipo de acción a ejecutar, la estructura de la carga útil del mensaje (request) y la estructura de la respuesta (response). A continuación, se resume la relación entre mensajes, Requerimientos Funcionales (RF) y Requerimientos No Funcionales (RNF):

Servicio Destino	Acción (Campo action)	RF Satisfecho	RNF Ma- peado	Protocolo y Puerto
autho	<i>Login (Implícito)</i>	RF-007, RF-008	Seguridad (RBAC), JWT	TCP Socket, Puerto 5000
inven	reserve	RF-002	Consistencia Estricta, Bloqueo Pesimista	TCP Socket, Puerto 5000
inven	commit	RF-004, RF-006	Integridad Transaccional, Auditoría	TCP Socket, Puerto 5000
inven	stock	RF-001, RF-003	Baja Latencia (< 200 ms)	TCP Socket, Puerto 5000

Cuadro 4: Resumen de contratos de mensajería TCP de la arquitectura SOA.

7.1. Detalle de Contratos de Mensajería TCP

A continuación se especifican las estructuras de las tramas de petición y de respuesta enviadas a través del ESB en formato JSON mediante sockets TCP crudos. Cabe destacar que todas las respuestas de los servicios inician con un prefijo de estado de dos caracteres (OK o NK) para indicar el éxito o fracaso a nivel del bus de servicios, seguido de la carga útil correspondiente.

7.1.1. 1. Servicio de Autenticación (Servicio auth)

Identificador en el Bus: autho

Descripción: Valida las credenciales del operario y devuelve un token firmado (JWT) con su rol asociado para habilitar el control de acceso en los clientes de frontera.

Estructura del Mensaje de Petición (Request TCP Payload):

```
{
  "username": "fillanca",
  "password": "mi_clave_segura_123"
}
```

Mensaje de Respuesta Exitoso (TCP Response Payload - Prefijo OK): La respuesta es enviada por el socket con el código de estado OK prepndido en la trama:

```
OK{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cGE6bnVudC5...[truncado]",
  "rol": "Administrador",
  "username": "fllancao"
}
```

Mensaje de Respuesta de Error (TCP Response Payload - Prefijo NK): En caso de fallo en las credenciales, el bus retorna la señal de error NK y la causa del rechazo:

NKCREDSNTIALS_INVALID

7.1.2. 2. Reserva Temporal de Stock (Servicio inven, Acción reserve)

Identificador en el Bus: inven

Acción: reserve

Descripción: Adquiere un bloqueo pesimista en base de datos y reserva temporalmente unidades de un producto en una ubicación específica para evitar la sobreventa.

Estructura del Mensaje de Petición (Request TCP Payload):

```
{
  "action": "reserve",
  "sku": "PES-CA1",
  "cantidad": 2,
  "ubicacion": "Tienda Valdivia",
  "usuarioId": "e4b2d288-43ea-46b2-a5e2-88229988ff22"
}
```

Mensaje de Respuesta Exitoso (TCP Response Payload - Prefijo OK):

```
OK{
  "reservaId": 12,
  "estado": "Pendiente",
  "expiraEn": "2026-06-03T01:25:00.000Z"
}
```

Mensaje de Respuesta de Error (TCP Response Payload - Prefijo NK):

NKSTOCK_INSUFICIENTE

7.1.3. 3. Confirmación de Reserva (Servicio inven, Acción commit)

Identificador en el Bus: inven

Acción: commit

Descripción: Transforma una reserva temporal en una venta confirmada definitiva, re-bajando definitivamente el stock reservado del catálogo.

Estructura del Mensaje de Petición (Request TCP Payload):

```
{
  "action": "commit",
  "reservaId": "b1b88220-432a-4632-a5e2-88229988ff22",
  "usuarioId": "e4b2d288-43ea-46b2-a5e2-88229988ff22"
}
```

Mensaje de Respuesta Exitoso (TCP Response Payload - Prefijo OK):

```
OK{
  "id": "c8e228c2-482a-466d-88b1-2299aaffd883",
  "total": 119980.00,
  "estado": "Pagada"
}
```

Mensaje de Respuesta de Error (TCP Response Payload - Prefijo NK):

NKRESERVATION_NOT_FOUND

7.1.4. 4. Consulta de Stock en Tiempo Real (Servicio inven, Acción stock)

Identificador en el Bus: inven

Acción: stock

Descripción: Devuelve la disponibilidad detallada de un SKU en todas las ubicaciones físicas y bodegas registradas.

Estructura del Mensaje de Petición (Request TCP Payload):

```
{
  "action": "stock",
  "sku": "PES-CA1"
}
```

Mensaje de Respuesta Exitoso (TCP Response Payload - Prefijo OK):

```
OK{
  "sku": "PES-CA1",
  "nombre": "Caña de Pescar Shakespeare Ugly Stik",
  "precio": 59990.00,
  "inventario": [
    {
      "bodega": "Tienda Valdivia",
      "tipo": "Tienda",
      "disponible": 10,
      "reservado": 0,
      "enTransito": 0
    },
    {
      "bodega": "Bodega E-commerce",
      "tipo": "Bodega",
      "disponible": 30,
      "reservado": 0,
      "enTransito": 0
    }
  ]
}
```

8. Documentación de Servicios y Procesos de Mensajería TCP

Para soportar las nuevas funcionalidades e integraciones de la 3ra Entrega, se ha extendido el catálogo de contratos de mensajería TCP del sistema Sync-Stock. Las peticiones externas son recibidas por el API Gateway perimetral de Express (actuando como un Protocol Bridge externo), el cual decodifica y valida las credenciales y el token JWT, evalúa el

control de acceso por rol (RBAC) con el middleware `verificarToken` y `permitirRoles`, empaqueta el contenido y lo retransmite al Enterprise Service Bus (ESB) por sockets TCP en formato JSON.

A continuación se detallan las operaciones adicionales incorporadas al catálogo de servicios del bus TCP:

8.1. Registro de Nuevos Productos

Servicio Destino: sales

Acción (Campo action): create-product

Descripción: Registra un nuevo producto en el catálogo y de forma automática crea un registro de inventario con stock inicial en todas las ubicaciones y bodegas disponibles bajo una única transacción ACID.

Estructura del Mensaje de Petición (Request TCP Payload):

```
{
  "action": "create-product",
  "sku": "PES-SE-NZ-023",
  "nombre": "Señuelo Rapala Magnum",
  "precio": 12500,
  "initialStock": 20
}
```

Mensaje de Respuesta Exitoso (TCP Response Payload - Prefijo OK):

```
OK{
  "id": "f5b88220-432a-4632-a5e2-88229988ffaa",
  "sku": "PES-SE-NZ-023",
  "nombre": "Señuelo Rapala Magnum",
  "precio": 12500
}
```

Mensaje de Respuesta de Error (TCP Response Payload - Prefijo NK):

NKPRODUCT_ALREADY_EXISTS

8.2. Modificación de Producto

Servicio Destino: sales

Acción (Campo action): update-product

Descripción: Actualiza los atributos básicos (nombre y precio) de un producto existente identificado por su SKU en el catálogo.

Estructura del Mensaje de Petición (Request TCP Payload):

```
{
  "action": "update-product",
  "sku": "PES-SE-NZ-023",

```

```
"nombre": "Señuelo Rapala Magnum Pro",  
"precio": 13990  
}
```

Mensaje de Respuesta Exitoso (TCP Response Payload - Prefijo OK):

```
OK{  
  "sku": "PES-SE-NZ-023",  
  "nombre": "Señuelo Rapala Magnum Pro",  
  "precio": 13990  
}
```

Mensaje de Respuesta de Error (TCP Response Payload - Prefijo NK):

NKPRODUCT_NOT_FOUND

8.3. Despacho de Órdenes Consolidadas

Servicio Destino: sales

Acción (Campo action): dispatch-order

Descripción: Marca una orden pagada y reservada temporalmente como despachada físicamente, actualizando su estado interno y liberando la reserva del flujo activo.

Estructura del Mensaje de Petición (Request TCP Payload):

```
{  
  "action": "dispatch-order",  
  "orderId": "d3b07384-d113-49c5-a5d6-d68aee460183"  
}
```

Mensaje de Respuesta Exitoso (TCP Response Payload - Prefijo OK):

```
OK{  
  "id": "d3b07384-d113-49c5-a5d6-d68aee460183",  
  "canal": "E_Commerce",  
  "estado": "Despachada",  
  "fechaCreacion": "2026-06-03T19:30:00.000Z",  
  "total": 12500  
}
```

Mensaje de Respuesta de Error (TCP Response Payload - Prefijo NK):

NKORDER_NOT_FOUND

8.4. Redirección General a través del Bus de Servicios (Acciones de Lectura)

Para dar cumplimiento estricto con el diseño arquitectónico SOA definido para PescaTotal, se definieron contratos TCP específicos para la consulta de información de solo lectura en el servicio **sales**. Cuando el API Gateway perimetral recibe las peticiones HTTP GET correspondientes, realiza la delegación enviando tramas TCP con las siguientes acciones al bus de servicios:

- **Consulta de Catálogo (action: 'products')**: Recupera la lista completa de productos junto con el stock disponible y reservado por ubicación.
- **Historial de Órdenes (action: 'orders')**: Recupera las últimas 15 órdenes de venta registradas en el sistema.
- **Listado de Reservas (action: 'reservations')**: Recupera el estado de las últimas 10 reservas de stock realizadas por el e-commerce.
- **Auditoría de Movimientos (action: 'history')**: Recupera los últimos 10 eventos de inventario registrados para control de auditoría.

De esta forma, el servidor Express actúa únicamente como un puente de red y traductor perimetral, mientras que el procesamiento y acceso a datos se centraliza en el bus de servicios TCP, manteniendo el desacoplamiento físico y de persistencia de la capa de negocio.

9. Documentación del Sistema y Evidencias de Funcionamiento

9.1. Explicación Técnica del Código e Implementación

El sistema Sync-Stock se implementó utilizando Node.js, TypeScript y Prisma, logrando los objetivos mediante los siguientes componentes core:

1. **Bus de Servicios TCP (ESB Customizado):** Implementado en `soabus.ts`, este módulo levanta un socket TCP en el puerto 5000. Los microservicios de inventario (`inven`), ventas (`sales`) y autenticación (`autho`) se registran abriendo sockets persistentes. La mensajería interna utiliza un protocolo de entramado customizado de longitud fija (Length-Prefixed Framing). Cada mensaje enviado posee un prefijo de 5 caracteres numéricos que especifican el tamaño exacto del contenido en bytes, seguido de 5 caracteres que identifican al servicio de destino. Esto previene problemas clásicos de fragmentación de buffer en streams TCP socket concurrentes.
2. **Control de Concurrencia y Bloqueos Pesimistas:** Para garantizar que no ocurran sobreventas en checkout simultáneos (POS físico y web), el servicio de inventario (`inventory.ts`) utiliza transacciones pesimistas (`prisma.$transaction`). Dentro de la transacción, se ejecuta una consulta raw SQL con `SELECT * FROM Inventario... FOR UPDATE`. Esto bloquea temporalmente las filas de stock específicas de los productos requeridos, forzando a que cualquier transacción concurrente deba esperar a que la transacción actual confirme o aborte.
3. **Prevención de Deadlocks:** Para evitar el problema de bloqueo mutuo (*deadlocks*) donde dos transacciones concurrentes retienen recursos cruzados, se incorporó un algoritmo de ordenamiento determinista en la función de checkout. Antes de adquirir los bloqueos en base de datos, el backend ordena el array de productos por su UUID de forma ascendente. Al bloquear los recursos siempre en el mismo orden físico en el disco, se imposibilita la formación de ciclos de espera.
4. **Daemon de Expiración (TTL) en Background:** En `backend/src/index.ts` se configuró un proceso daemon en segundo plano que corre cada 10 segundos. Este daemon escanea la base de datos de manera atómica buscando registros en la tabla `Reserva` que estén en estado `Pendiente` y cuya `fechaExpiracion` sea menor al tiempo actual. Al detectar expiraciones, abre una transacción que revierte las unidades reservadas devolviéndolas al stock vendible disponible y actualiza el estado a `Expirada`.

9.2. Evidencias de Funcionamiento y Logs

A continuación se adjuntan las trazas de ejecución en consola obtenidas directamente del servidor de backend de Sync-Stock, demostrando el ciclo de vida de los servicios, el control de concurrencia y las tareas en segundo plano.

Logs de Inicialización del ESB y Conexión de Servicios:

```
[ESB-TCP] Server listening on port 5000...
[ESB-TCP] New client connection established.
[ESB-TCP] Registered service: 'autho' on local socket.
[ESB-TCP] Registered service: 'inven' on local socket.
[ESB-TCP] Registered service: 'sales' on local socket.
[ESB-Mediator] ESB Mediator registering services via TCP sockets...
[ESB-Mediator] Services successfully mapped and listening for requests.
[Express-Gateway] Gateway API listening on port 3001.
```

Logs de Transacción de POS y Bloqueo Pesimista (Checkout Concurrente):

```
[Express-Gateway] POST /api/v1/sales/checkout received from POS
[ESB-TCP] Routing message to service 'sales' (action: 'checkout')
[Sales-Service] Processing checkout. Total items: 2.
[Inventory-Service] Initiating stock check and lock transaction.
[Inventory-Service] Sorted product list to prevent deadlocks:
  - SKU: PES-CANA-CARBON-01 (UUID: a1b2c3d4-...)
  - SKU: PES-SE-NZ-023 (UUID: f5b88220-...)
[Prisma-SQL] SELECT * FROM "Inventario" WHERE ... FOR UPDATE
[Prisma-SQL] Row locks acquired successfully. No contention.
[Inventory-Service] Stock verified. Deducting 2 units.
[Prisma-SQL] UPDATE "Inventario" SET "stockDisponible" = 13 WHERE ...
[Sales-Service] Order d3b07384-d113... recorded successfully.
[Inventory-Service] Transaction committed. Locks released.
[ESB-TCP] Transaction checkout response returned to Protocol Bridge.
[Express-Gateway] 201 Created returned to POS vendor.
```

Logs del Daemon de Expiración Temporal de Reservas Web (TTL):

```
[Daemon-TTL] Scanning database for expired web reservations...
[Daemon-TTL] Found 1 expired pending reservation (ID: b1b88220-432a...).
[Inventory-Service] Initiating reservation rollback transaction.
[Prisma-SQL] SELECT * FROM "Inventario" WHERE ... FOR UPDATE
[Inventory-Service] Reverting 2 units of SKU PES-CANA-CARBON-01.
[Prisma-SQL] UPDATE "Inventario" SET
  "stockDisponible" = "stockDisponible" + 2,
  "stockReservado" = "stockReservado" - 2 WHERE ...
[Prisma-SQL] UPDATE "Reserva" SET "estado" = 'Expirada' WHERE ...
[Inventory-Service] Rollback transaction committed successfully.
[Daemon-TTL] Expired reservation b1b88220 reclaimed. Stock restored.
```

10. Análisis Crítico

10.1. Latencia y Concurrencia de los Bloqueos Pesimistas

El uso de bloqueos pesimistas (`SELECT ... FOR UPDATE`) garantiza una consistencia absoluta (consistencia estricta en el contexto del teorema de CAP), lo cual es crítico para erradicar las sobreventas en PescaTotal. Sin embargo, esta decisión de diseño conlleva un costo en latencia bajo escenarios de alta concurrencia. Cuando se bloquean filas de inventario para un producto altamente demandado (por ejemplo, ofertas flash en el canal web), las peticiones de checkout concurrentes se encolan secuencialmente en el motor PostgreSQL, aumentando la latencia general del sistema de < 50 ms hasta picos de > 1000 ms. Para mitigar esto en un entorno productivo de mayor escala, se debería evaluar una estrategia híbrida utilizando optimismo de stock mediante caches en memoria (ej. Redis) o reservas distribuidas desacopladas del motor de base de datos relacional.

10.2. Desafíos del Desarrollo de un ESB Personalizado TCP

La decisión de implementar un bus de servicios personalizado sobre sockets TCP crudos en Node.js permitió lograr un sistema ultraligero y sin dependencias de infraestructura externas. No obstante, en un entorno de desarrollo real esto introduce complejidades de bajo nivel que los brokers industriales (como RabbitMQ, Apache Kafka o gRPC) resuelven de forma nativa:

- **Entramado de Datos y Fragmentación:** Al usar streams TCP crudos, los mensajes pueden llegar fragmentados o concatenados si se transmiten de forma muy veloz. Se tuvo que programar un algoritmo de parseo manual con prefijo de longitud para solucionar esto de forma segura.
- **Gestión de Colas y Reintentos:** A diferencia de RabbitMQ, nuestro bus customizado no almacena físicamente los mensajes en disco (encolamiento persistente) ni cuenta con mecanismos de Dead Letter Queues (DLQ) para reenvío de mensajes fallidos de forma automática.

10.3. Seguridad y RBAC en Entornos Distribuidos

El diseño e implementación de un middleware de control de acceso por roles (RBAC) centralizado en el Gateway REST protege con éxito la capa lógica del ESB de peticiones externas no autorizadas. Los usuarios del sistema solo interactúan con pantallas permitidas (por ejemplo, restringiendo la pestaña de Catálogo CRUD y la Auditoría de mensajería del ESB al administrador). Sin embargo, a nivel de red interna, la comunicación TCP entre el Protocol Bridge (Express) y el bus de servicios viaja actualmente en texto plano. En un entorno de producción distribuido real en la nube, es imperativo implementar encriptación de extremo a extremo mediante el módulo `tls` de Node.js, configurando certificados SSL/TLS autofirmados para mitigar ataques de escucha de red interna (Man-in-the-Middle).

10.4. Control de Inactividad y Seguridad Física en Tienda

La incorporación de la autodetección de inactividad de 30 minutos es un paso crítico para proteger la seguridad de la información física en la tienda PescaTotal. Dado que las

terminales del Punto de Venta (POS) suelen compartirse entre diferentes vendedores en el mesón de atención física, un olvido de cierre de sesión podría permitir a un vendedor registrar ventas a nombre de otro o a un cliente acceder a configuraciones de stock del panel administrador. La simulación de inactividad implementada proporciona un mecanismo de auditoría rápido que valida que el estado reactivo del frontend se limpie correctamente al expirar la sesión.

11. Conclusiones

Con la definición del modelo de datos relacional y la estructuración en base a servicios (SOA), el sistema **Sync-Stock** adquiere la solidez técnica necesaria para erradicar la sobreventa en PescaTotal. La separación entre componentes cliente y servicio asegura que la plataforma sea escalable, permitiendo a futuro integrar nuevos canales de venta sin alterar el núcleo del control de inventario. La estricta vinculación de cada interfaz con los requerimientos de seguridad, consistencia y baja latencia asegura el cumplimiento tanto de los objetivos comerciales como de los estándares de ingeniería requeridos.